Agent 记忆与 Skill

上下文文件、SKILL.md 和 system 段

Agent 记忆与 Skill

- 背景
- 目标与非目标
- 本模块在仓库中的位置
- 接入 agent 循环
- 核心数据结构
  - 发现顺序
  - system 段
  - 显式块
  - 上下文文件优先级
- 方案
- 备选方案
- 横切关注点
- 手搓实现笔记

Agent 记忆与 Skill

背景

上游没有向量记忆。模型跨轮能用的持久文本是会话 JSONL（见 压缩与中断恢复）、上下文文件、以及 skill 索引。索引的实现是 `packages/coding-agent/src/core/skills.ts` 的 `Skill`、`loadSkills`、`formatSkillsForPrompt`，上下文文件是 `core/resource-loader.ts` 的 `loadContextFileFromDir` / `loadProjectContextFiles`，拼进 prompt 的是 `core/system-prompt.ts` 的 `buildSystemPromptSections`。

`@my/resources` 只做文件扫描和字符串。它不订阅 `Agent`，也不写 JSONL。

目标与非目标

目标

- 每次请求的 system 段由纯函数生成，测试可以不启动 `Agent`。
- skill 正文默认不进 prompt。模型用 M4 的 `read` 按 `location` 去读。

非目标

- 不实现 embedding、跨会话自动回忆、`allowed-tools` 自动放权。
- 不把 skill 实现成可执行插件。插件是 网关 的扩展。

本模块在仓库中的位置

```
packages/resources/
  package.json                # 无 @my/* 依赖
  tsconfig.json
  src/index.ts
  src/paths.ts                # agentDir()。上游 config.ts 的 getAgentDir
  src/skills.ts               # 上游 core/skills.ts，外加 parseSkillBlock
  src/context-files.ts        # 上游 resource-loader.ts 的 loadProjectContextFiles
  src/system-prompt.ts        # 上游 core/system-prompt.ts
  test/skills.test.ts
  test/context-files.test.ts
  test/system-prompt.test.ts
```

文件	职责
`paths.ts`	解析 `MY_AGENT_DIR`，默认 `path.join(os.homedir(), ".my-agent")`
`context-files.ts`	从 cwd 向上收集每目录至多一个上下文文件
`skills.ts`	校验 frontmatter、扫描目录、格式化索引、解析显式 skill 块
`system-prompt.ts`	把上下文和 skill 索引放进命名段

依赖：禁止 import `@my/*`。`apps/cli/src/wire.ts` 调用这些函数，再把字符串交给 `AgentSession`。

磁盘

```
~/.my-agent/skills/**/SKILL.md
~/.agents/skills/**/SKILL.md
<cwd>/.my-agent/skills/**/SKILL.md          # 仅 projectTrusted
<cwd> 向上直到 git 根的 .agents/skills/      # 仅 projectTrusted
<dir>/AGENTS.override.md | AGENTS.md | AGENTS.MD | CLAUDE.md | CLAUDE.MD
```

git 根的判定放到 M4：`context-files.ts` 的选项 `stopAt?: string`。M3 的 `stopAt` 由调用方传入。`wire.ts` 在 M4 之前传 cwd 的祖先里第一个含 `.git` 的目录；若还没有 `findGitPaths`，就向上走到文件系统根。M4 再改成 `findGitPaths(cwd)?.repoDir` 的父链停止点，与上游 `findGitRepoRoot` 一致：看到 `.git` 文件或目录就停。

接入 agent 循环

`wire.ts` 在每次 `AgentSession.prompt` 之前：

1. `loadProjectContextFiles` 和 `loadSkills`。
2. `buildSystemPromptSections`。

3. 若生成的段与当前 `agent.state` 首条 system 消息的 `sections` 不同，构造一条 `SystemMessage`：`content: ""`，`sections` 为新段，`timestamp: Date.now()`，交给 session 追加。

不要在 `runLoop` 里读磁盘。skill 正文进入上下文的唯一路径是模型调用 `read`，结果成为 `toolResult`。

显式命令由 CLI 处理，不由 resources 注册 slash command。用户输入以 `/skill:` 开头时，CLI 读文件并调用 `prompt`，消息正文用下面的 `parseSkillBlock` 能识别的格式。

核心数据结构

```ts
export interface ContextFile {
  path: string;     // 绝对路径
  content: string;  // 已去掉 BOM
}

export interface SkillFrontmatter {
  name?: string;
  description?: string;
  "disable-model-invocation"?: boolean;
}

export interface Skill {
  name: string;
  description: string;
  filePath: string;   // SKILL.md 的绝对路径
  baseDir: string;    // SKILL.md 的父目录
  scope: "user" | "project" | "path";
  disableModelInvocation: boolean;
}

export interface ResourceDiagnostic {
  type: "warning" | "collision";
  message: string;
  path: string;
  collision?: { name: string; winnerPath: string; loserPath: string };
}

export interface LoadSkillsResult {
  skills: Skill[];
  diagnostics: ResourceDiagnostic[];
}

export interface LoadSkillsOptions {
  cwd: string;
  agentDir: string;
  projectTrusted: boolean;
  /** 额外文件或目录，在默认目录之后加载，因此同名会输给默认目录 */
  extraPaths?: string[];
}

export interface SystemPromptSections {
  preamble: string;
  tools?: string;
  rules?: string;
  project_context?: string;
  skills?: string;
  cwd: string;
}

export interface ParsedSkillBlock {
  name: string;
```

```
  location: string;
  content: string;
  userMessage?: string;
}
```

`SkillFrontmatter` 对应上游 `skills.ts` 的同名接口。未知 YAML 键忽略，不报错。

名字不变量，与 `validateName` 相同：

- 长度 1 到 64。
- 匹配 `^[a-z0-9-]+$`。
- 不以 `-` 开头或结尾，不含 `--`。

description 必须是非空字符串且长度 ⩽ 1024。不合规则不产生 `Skill`，产生 `diagnostics` 一条，`type: "warning"`。

发现顺序

`loadSkills` 按这个数组依次 `add`。同名保留先加入的，后加入的记 `collision`，`winnerPath` 是先加入的 `filePath`。

顺序	目录	条件	scope
1	`<cwd>/.my-agent/skills` 以及向上直到 `stopAt` 的每一层 `.my-agent/skills`	`projectTrusted`	`project`
2	同样路径上的 `.agents/skills`	`projectTrusted`	`project`
3	`<agentDir>/skills`	总是	`user`
4	`~/.agents/skills`	总是	`user`
5	`extraPaths`	总是	`path`

同一目录内按文件名字母序扫描，保证测试稳定。这是手搓默认。上游目录顺序依赖文件系统，不适合做验收断言。

扫描规则来自 `loadSkillsFromDir`：

- 子目录含 `SKILL.md` 时，它是 skill 根，不再进入其子目录。
- 不含 `SKILL.md` 的目录继续向下。
- 根目录直接放着的 `*.md` 也当成一个 skill，`name` 默认是文件名去掉 `.md`。这种文件仍要有 frontmatter description。
- 跳过匹配 `.gitignore` 的路径。M3 只认技能根上的 `.gitignore`，不实现上游那套逐级前缀。默认写进完成定义：只读 skill 根目录的 `.gitignore`。

system 段

```
export function buildSystemPromptSections(input: {
  cwd: string;
  contextFiles: ContextFile[];
```

```
  skills: Skill[];
  hasFileReader: boolean;
  toolNames: string[];
}): SystemPromptSections
```

- `preamble` 固定为 `You are a coding assistant.`。不要复制上游那整段 Pi 自我介绍。
- `tools` 是 `toolNames` 每行一个 `- name`。
- `project_context` 在 `contextFiles.length > 0` 时存在。每个文件包一层 `<project_instructions path="绝对路径">`。
- `skills` 只在 `hasFileReader && formatSkillsForPrompt 非空` 时存在。
- `cwd` 把反斜杠换成 `/`。

`formatSkillsForPrompt` 跳过 `disableModelInvocation`。输出形状：

```
The following skills provide specialized instructions for specific tasks.
Use the read tool to load a skill file when the task matches its description.

<available_skills>
  <skill>
    <name>pdf-tools</name>
    <description>...</description>
    <location>/abs/SKILL.md</location>
  </skill>
</available_skills>
```

XML 转义 `& < > " '`。没有可见 skill 时返回 `""`。

显式块

`parseSkillBlock` 使用这个正则，与上游 `agent-session.ts` 的 `parseSkillBlock` 相同：

```
/^<skill name="([^"]+)" location="([^"]+)">\n([\s\S]*?)\n<\/skill>(?:\n\n([\s\S]+))?$/
```

CLI 构造消息时用：

```
export function renderSkillInvocation(skill: Skill, body: string, userText: string | undefined): string {
  const tail = userText && userText.trim() ? `\n\n${userText.trim()}` : "";
  return `<skill name="${skill.name}" location="${skill.filePath}">\n${body}\n</skill>${tail}`;
}
```

`body` 是 `SKILL.md` 去掉 frontmatter 之后的 Markdown。

上下文文件优先级

同一目录只取第一个存在且是普通文件的名字：

```
const CONTEXT_FILENAMES = ["AGENTS.override.md", "AGENTS.md", "AGENTS.MD", "CLAUDE.md", "CLAUDE.MD"] as const;
```

从 `cwd` 向上，每层最多一个文件，直到 `stopAt` 那一层（包含）。再额外读 `<agentDir>` 根上的一个上下文文件，若有的话放在数组最后。不要读 `<agentDir>/skills` 里的 Markdown 当上下文。

`shadowedPath?: string`：若某个 `ContextFile.path` 等于它，从结果里去掉。M3 不计算这个路径。M4 的 `wire.ts` 传入主仓被 worktree 挡住的那个绝对路径。

方案

项目信任是一个布尔值，存在 `apps/cli` 的内存里，M3 默认 `true` 当 cwd 下存在 `.my-agent/` 或 `.git` 由环境变量 `MY_AGENT_TRUST=1` 强制，否则 `false`。不要做交互式询问。未信任时上下文文件仍然加载（与上游 configuration 文档一致），skill 的项目目录不加载。

`/reload` 在 M3 就是再调用一次上面的三个函数并追加 system 消息。没有单独的 reload 模块。

备选方案

方案	放弃原因
启动时读入所有 `SKILL.md` 正文	索引才是上游的行为。全文会挤占 `keepRecentTokens`
resources 依赖 session 以便写 system 消息	依赖表禁止。字符串由 CLI 送进 session
同名 skill 后写的覆盖先写的	上游保留先发现的并报 collision。静默覆盖无法写测试

横切关注点

- **错误**：单个坏 skill 不影响其他 skill。读文件抛错时记 warning，`content` 不部分返回。
- **并发**：扫描是同步 `fs`。不要在工具执行中途调用 `loadSkills`。CLI 在 `session` 的 phase 为 `idle` 时才能 reload。
- **持久化**：skill 文件不复制进 JSONL。会话里只有当时的 system 段。文件改了之后，旧检查点保持旧文本，直到下一次 prompt 追加新的 system 消息。
- **安全**：`projectTrusted === false` 时项目 skill 目录的 `readdir` 次数为 0。测试用计数或临时目录断言结果里没有那些路径。

手搓实现笔记

按总览 M3 的顺序。先写夹具目录再写扫描器。`system-prompt.test.ts` 对一段固定的 `Skill` 对象做快照，快照包含 `<location>` 且不包含正文里的句子。`parseSkillBlock` 的失败用例是普通用户句子，返回 `null`。